



23

Responding to User Gestures

All of the examples that you've implemented so far in this book have relied on user gestures. In traditional web applications, you mostly deal with mouse events. However, touchscreens rely on the user manipulating elements with their fingers, which is fundamentally different from the mouse.

In this chapter, first, you'll learn about scrolling. This is probably the most common gesture, besides touch. Then, you'll learn about giving the user the appropriate level of feedback when they interact with your components. Finally, you'll implement components that can be swiped.

The goal of this chapter is to show you how the **gesture response system** inside React Native works and some of the ways in which this system is exposed via components.

In this chapter, we'll cover the following topics:

- Scrolling with your fingers

- Giving touch feedback
- Using Swipeable and cancellable components

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter23>.

Scrolling with your fingers

Scrolling in web applications is done by using the mouse pointer to drag the scrollbar back and forth or up and down, or by spinning the mouse wheel. This doesn't work on mobile devices because there's no mouse. Everything is controlled by gestures on the screen.

For example, if you want to scroll down, you use your thumb or index finger to pull the content up by physically moving your finger over the screen.

Scrolling like this is difficult to implement, but it gets more complicated. When you scroll on a mobile screen, the velocity of the dragging motion is taken into consideration. You drag the screen fast, then let go, and the screen continues to scroll based on how fast you moved your finger. You can also touch the screen while this is happening to stop it from scrolling.

Thankfully, you don't have to handle most of this stuff. The `ScrollView` component handles much of the scrolling complexity for you. In fact, you've already used the `ScrollView` component back in *Chapter 20, Rendering Item Lists*. The `ListView` component has `ScrollView` baked into it.



You can adjust the low-level parts of user interactions by implementing gesture life cycle methods. You'll probably never need to do this, but if you're interested, you can read about it at

<https://reactnative.dev/docs/gesture-responder-system>.

You can use `ScrollView` outside of `ListView`. For example, if you're just rendering arbitrary content such as text and other widgets: not a list, in other words: you can just wrap it in `<ScrollView>`. Here's an example:

```
export default function App() {
  return (
    <View style={styles.container}>
      <ScrollView style={styles.scroll}>
        {new Array(20).fill(null).map((v, i) => (
          <View key={i}>
            <Text style={[styles.scrollItem, styles.text]}>Some text</Text>
            <ActivityIndicator style={styles.scrollItem} size="large" />
            <Switch style={styles.scrollItem} />
          </View>
        ))}
      </ScrollView>
    </View>
  )
}
```

```
        </View>
      )})
    </ScrollView>
  </View>
);
}
```

The `ScrollView` component isn't of much use on its own: it's there to wrap other components. It needs height in order to function correctly. Here's what the scroll style looks like:

```
scroll: {
  height: 1,
  alignSelf: "stretch",
},
```

The `height` property is set to `1`, but the stretch value of `alignSelf` allows the items to display properly. Here's what the end result looks like:

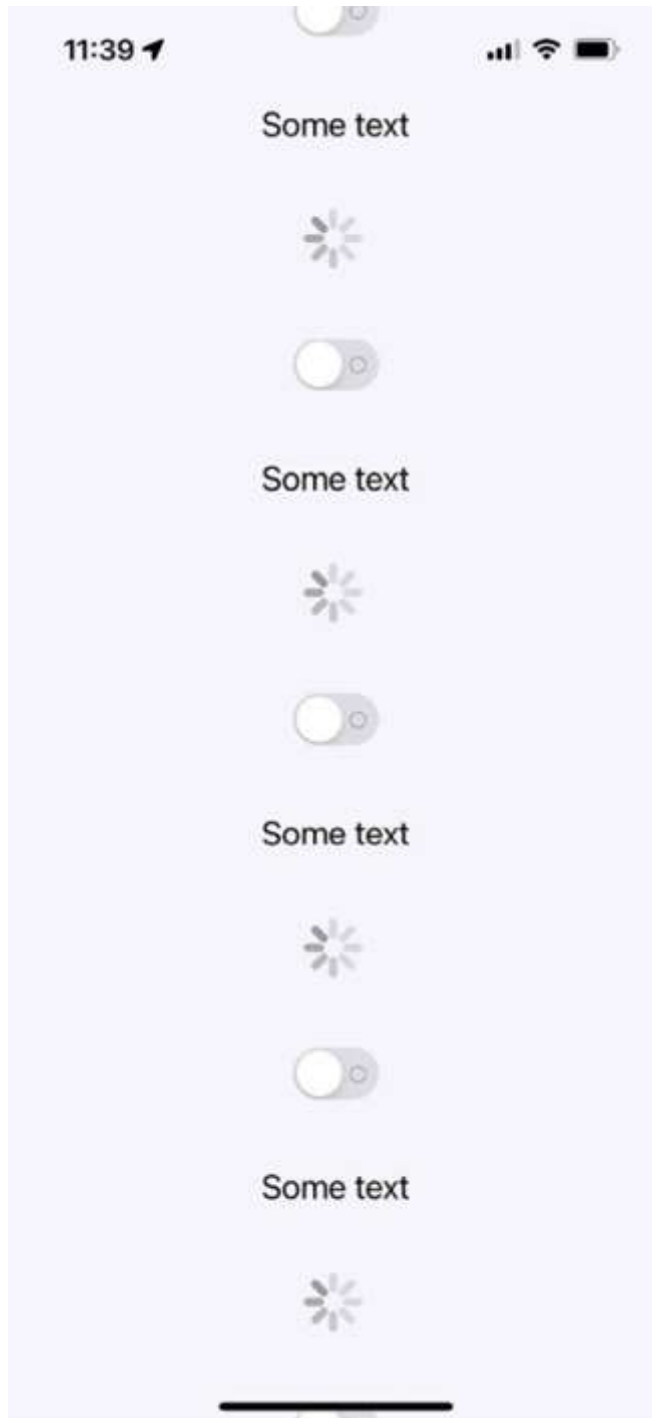


Figure 23.1: ScrollView

There's a vertical scrollbar on the right-hand side of the screen as you drag the content down. If you run this example, you can play around with making various gestures, such as making content scroll on its own and then making it stop.

When the user scrolls through content on the screen, they receive visual feedback. Users should also receive visual feedback when they touch certain elements on the screen.

Giving touch feedback

The React Native examples you've worked with so far in this book have used plain text to act as buttons or links. In web applications, to make text look like something that can be clicked, you just wrap it with the appropriate link. There's no link component in React Native, so you can style your text to look like a button.



The problem with trying to style text as links on mobile devices is that they're too hard to press. Buttons provide a bigger target for fingers, and they're easier to apply touch feedback on.

Let's style some text as a button. This is a great first step as it makes the text look touchable. But you also want to give visual feedback to the user when they start interacting with the button. React Native provides several components to help with this:

- `TouchableOpacity`
- `TouchableHighlight`
- Pressable API

But before diving into the code, let's take a look at what these components look like visually when users interact with them, starting with `TouchableOpacity`:

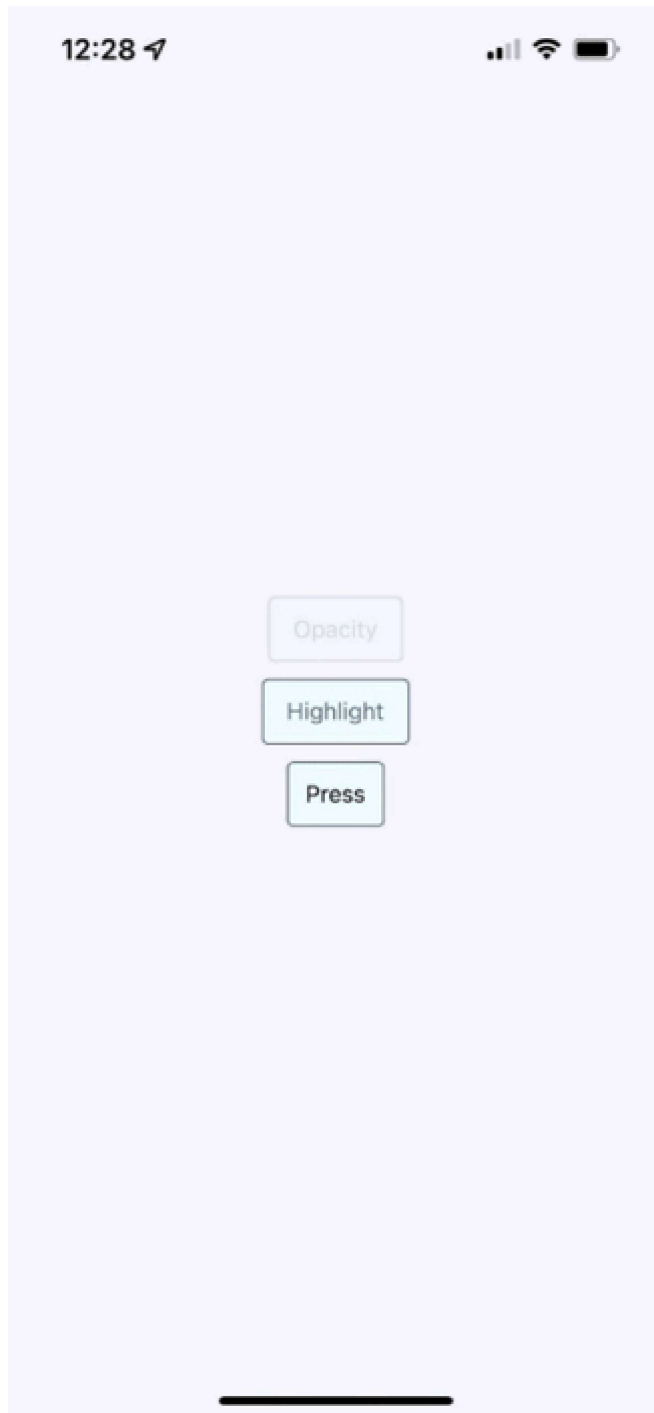


Figure 23.2: TouchableOpacity

There are three buttons being rendered here. The top one, labeled **Opacity**, is currently being pressed by the user. The opacity of the button is dimmed when pressed, which provides important visual feedback for the user.

Let's see what the **Highlight** button looks like when pressed:

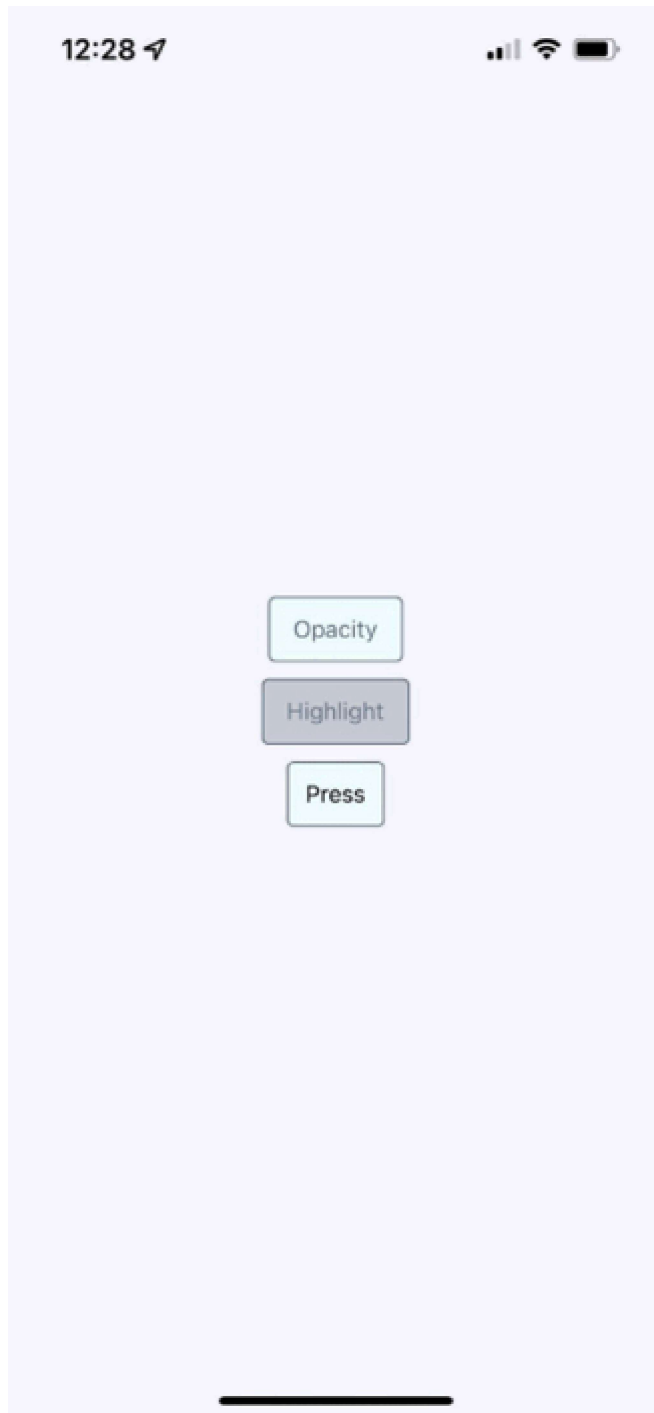


Figure 23.3: TouchableHighlight

Instead of changing the opacity when pressed, the `TouchableHighlight` component adds a highlight layer over the button. In this case, it's highlighted using a more transparent version of the slate gray that's being used in the font and border colors.

The last example of a button is provided by the `Pressable` component. The Pressable API has been introduced as a core component wrapper and allows different stages of press interaction on any of its defined children. With such components, we can handle `onPressIn`, `onPressOut` (which we explore in the next chapter), and `onLongPress` callbacks and implement any touchable feedback that we want. Let's take a look at how `PressableButton` looks when we click on it:

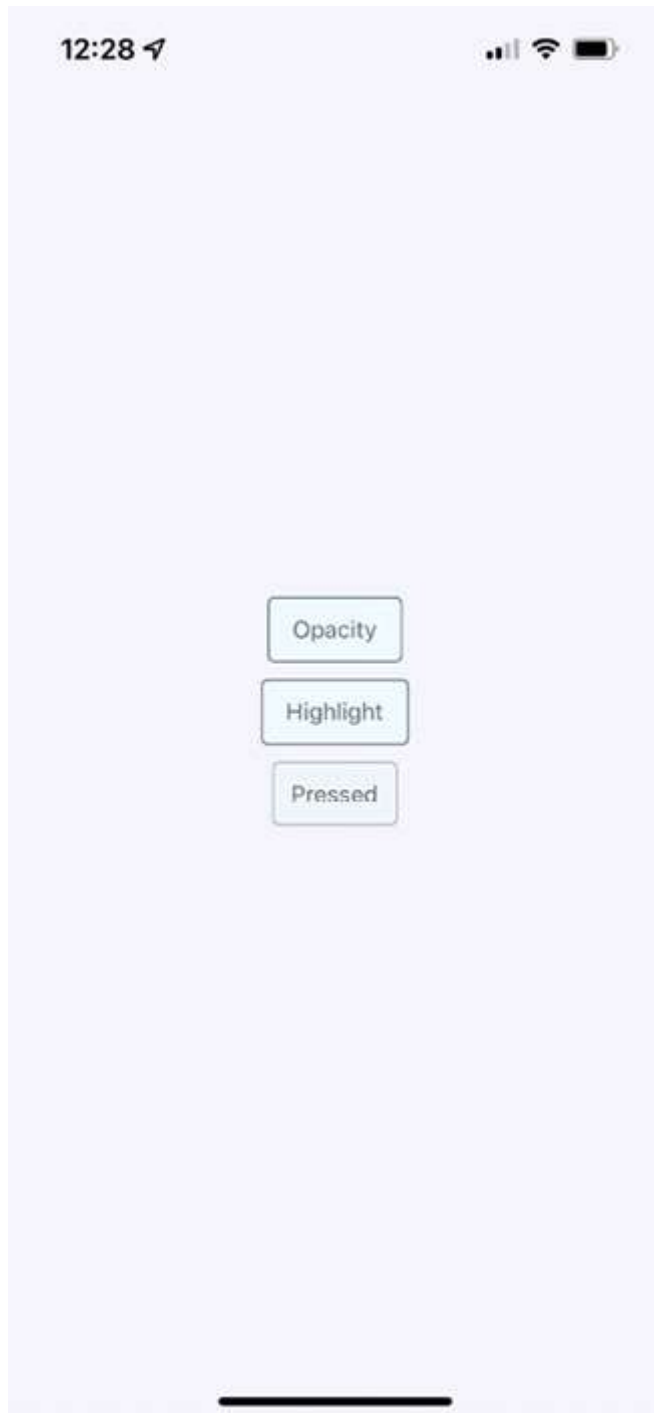


Figure 23.4: Pressable button

If we continue to keep our finger on this button, we get an `on-LongPress` event and the button will update:

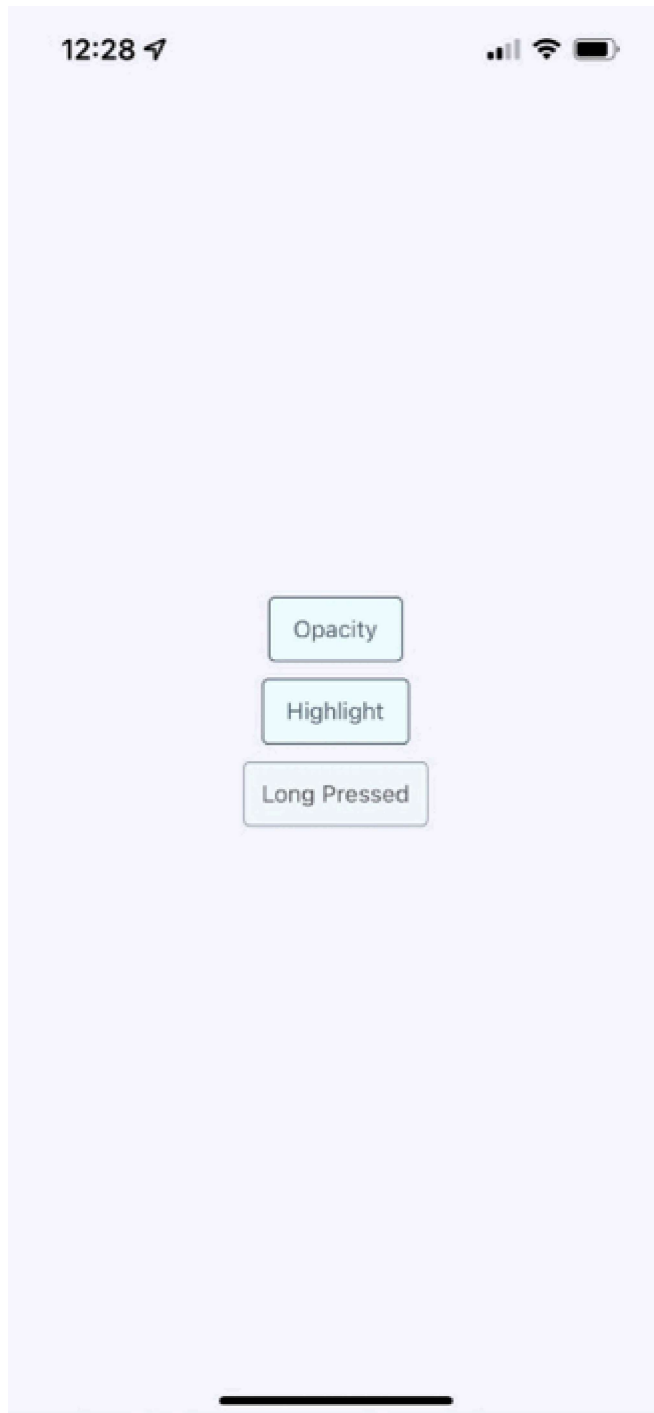


Figure 23.5: Long Pressed button

It doesn't really matter which approach you use. The important thing is that you provide the appropriate touch feedback for your users as they interact with your buttons. In fact, you might want to use all the approaches in the same app, but for different things.

Let's create an `OpacityButton` and `HighlightButton` component, which makes it easy to use the first two approaches:

```
type ButtonProps = {
  label: string;
  onPress: () => void;
};

export const OpacityButton = ({ label, onPress }: ButtonProps) => {
  return (
    <TouchableOpacity
      style={styles.button}
      onPress={onPress}
      activeOpacity={0.5}
    >
      <Text style={styles.buttonText}>{label}</Text>
    </TouchableOpacity>
  );
};

export const HighlightButton = ({ label, onPress }: ButtonProps) => {
  return (
    <TouchableHighlight
```

```
    style={styles.button}
    underlayColor="rgba(112,128,144,0.3)"
    onPress={onPress}
  >
    <Text style={styles.buttonText}>{label}</Text>
  </TouchableHighlight>
);
};
```

Here are the styles that were used to create this button:

```
button: {
  padding: 10,
  margin: 5,
  backgroundColor: "azure",
  borderWidth: 1,
  borderRadius: 4,
  borderColor: "slategrey",
},
buttonText: {
  color: "slategrey",
},
```

Now let's take a look at the button based on the Pressable API:

```
const PressableButton = () => {
  const [text, setText] = useState("Not Pressed");
  return (
```

```
<Pressable
  onPressIn={() => setText("Pressed")}
  onPressOut={() => setText("Press")}
  onLongPress={() => {
    setText("Long Pressed");
  }}
  delayLongPress={500}
  style={({ pressed }) => [
    {
      opacity: pressed ? 0.5 : 1,
    },
    styles.button,
  ]}
>
  <Text>{text}</Text>
</Pressable>
);
};
```

Here's how you can put those buttons into the main app module:

```
export default function App() {
  return (
    <View style={styles.container}>
      <OpacityButton onPress={() => {}} label="Opacity" />
      <HighlightButton onPress={() => {}} label="Highlight" />
      <PressableButton />
    </View>
  );
}
```

```
);  
}
```

Note that the `onPress` callbacks don't actually do anything: we're passing them because they're a required property.

In the following section, you'll learn about providing feedback when the user swipes elements across the screen.

Using Swipeable and Cancellable components

Part of what makes native mobile applications easier to use than mobile web applications is that they feel more intuitive. Using gestures, you can quickly get a handle on how things work. For example, swiping an element across the screen with your finger is a common gesture, but the gesture has to be discoverable.

Let's say that you're using an app, and you're not exactly sure what something on the screen does. So, you press down with your finger and try dragging the element. It starts to move. Unsure of what will happen, you lift your finger up, and the element moves back into place. You've just discovered how part of this application works.

You'll use the `Scrollable` component to implement **swipeable** and **cancellable** behaviors like this. You can create a somewhat generic component that allows the user to swipe text off the screen and, when that happens, call a callback function. Let's look at the code that will render the swipeables before we look at the generic component itself:

```
export default function SwipableAndCancellable() {
  const [items, setItems] = useState(
    new Array(10).fill(null).map((v, id) => ({ id, name: "Swipe Me" })))
  );
  function onSwipe(id: number) {
    return () => {
      setItems(items.filter((item) => item.id !== id));
    };
  }
  return (
    <View style={styles.container}>
      {items.map((item) => (
        <Swipeable
          key={item.id}
          onSwipe={onSwipe(item.id)}
          name={item.name}
          width={200}
        />
      ))}
    </View>
  );
}
```

```
);  
}
```

This will render 10 `<Swipeable>` components on the screen.

Let's see what this looks like:

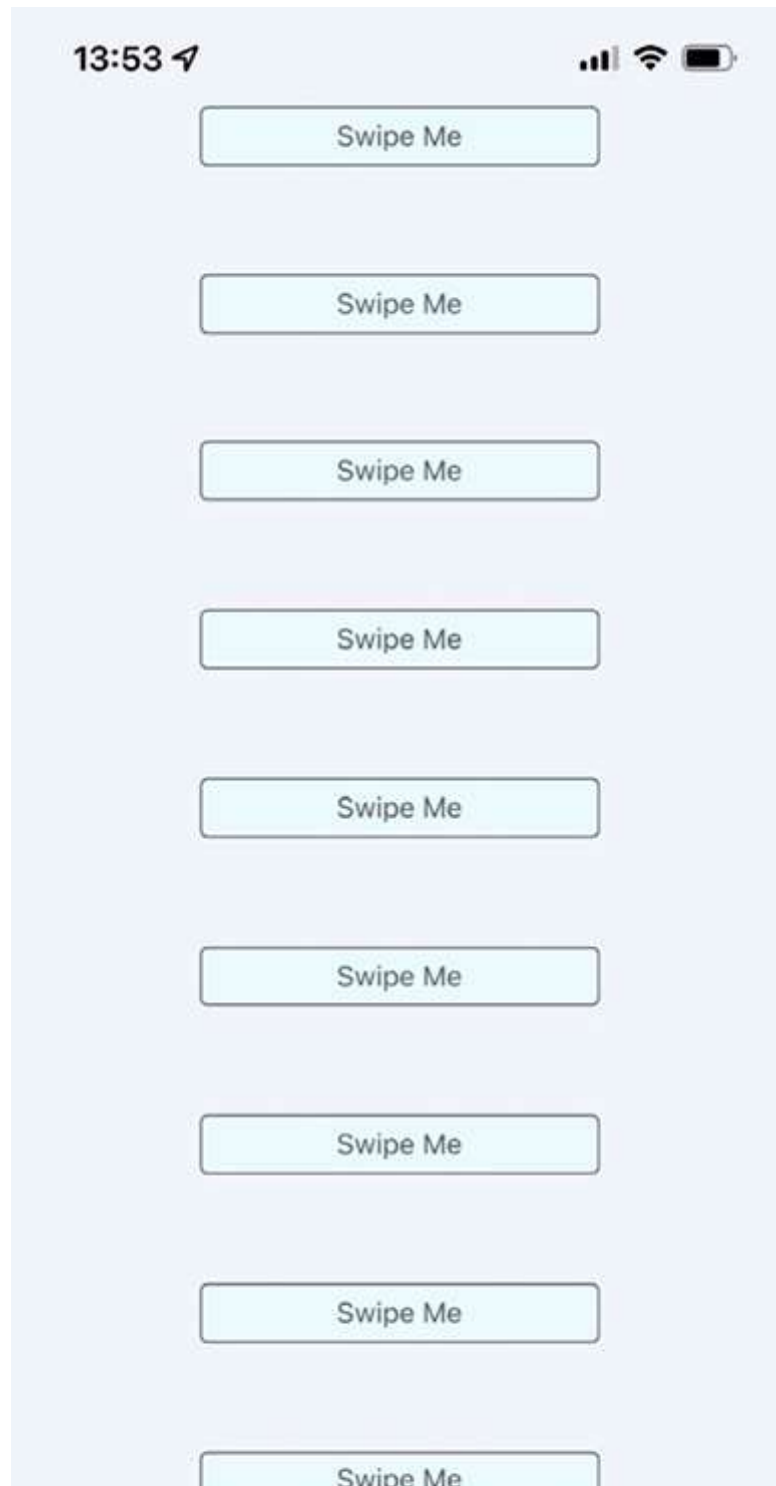




Figure 23.6: Screen with Swipeable components

Now, if you start to swipe one of these items to the left, it will move. Here's what this looks like:

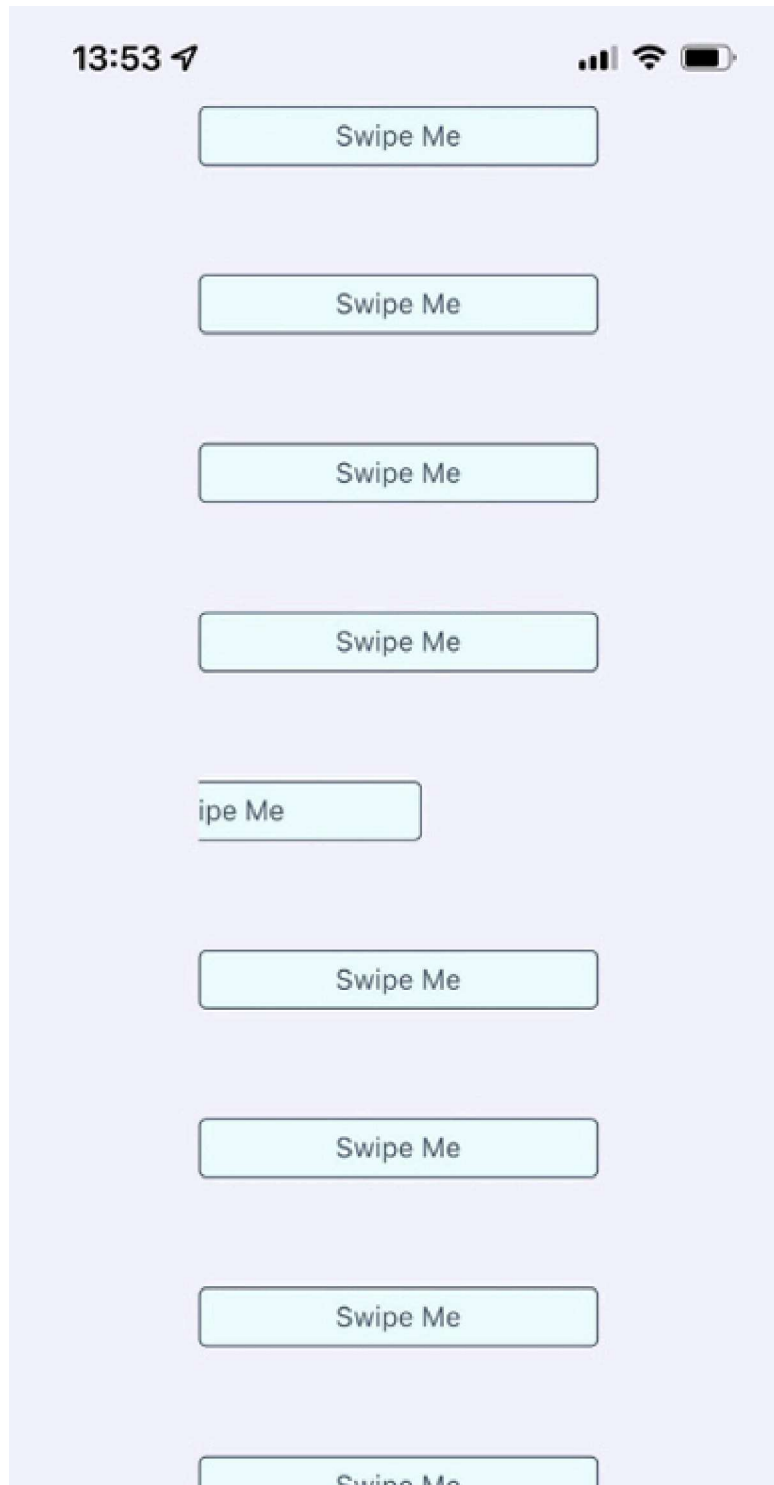
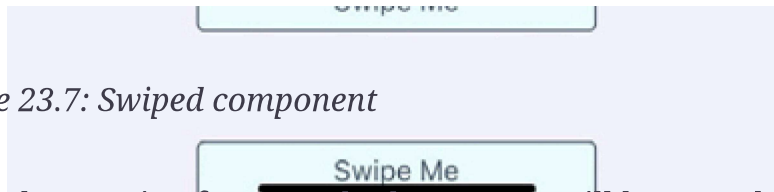


Figure 23.7: Swiped component

If you don't swipe far enough, the gesture will be canceled and the item will move back into place, as expected. If you swipe it all the way, the item will be removed from the list completely and the items on the screen will fill the empty space.

Now, let's take a look at the `Swipeable` component itself:

```
type SwipeableProps = {
  name: string;
  width: number;
  onSwipe: () => void;
};

export default function Swipeable({ name, width, onSwipe }: SwipeableProps) {
  function onScroll(e: NativeSyntheticEvent<NativeScrollEvent>) {
    console.log(e.nativeEvent.contentOffset.x);
    e.nativeEvent.contentOffset.x >= width && onSwipe();
  }
  return (
    <View style={styles.swipeContainer}>
      <ScrollView
        horizontal
        snapToInterval={width}
        showsHorizontalScrollIndicator={false}
        scrollEventThrottle={10}
        onScroll={onScroll}
      />
    </View>
  );
}
```

```
    <View style={[styles.swipeItem, { width }]}>
      <Text style={styles.swipeItemText}>{name}</Text>
    </View>
    <View style={[styles.swipeBlank, { width }]} />
  </ScrollView>
</View>
);
}
```

The component accepts the `width` property to specify the width itself, `snapToInterval` to create paging-like behavior with swipe canceling, and handling the distance where we can call the `onSwipe` callback to remove items from the list.

To enable swipe to the left, we need to add a blank component beside the component with text in it. Here are the styles that are used for this component:

```
swipeContainer: {
  flex: 1,
  flexDirection: "row",
  width: 200,
  height: 30,
  marginTop: 50,
},
swipeItem: {
  height: 30,
  backgroundColor: "azure",
```

```
justifyContent: "center",
borderWidth: 1,
borderRadius: 4,
borderColor: "slategrey",
},
swipeItemText: {
  textAlign: "center",
  color: "slategrey",
},
swipeItemBlank: {
  height: 30,
},
```

The `swipeItemBlank` style has the same height as `swipeItem`, but nothing else. It's invisible.

We have now covered all the topics in this chapter.

Summary

In this chapter, we were introduced to the idea that gestures on native platforms make a significant difference compared to mobile web platforms. We started off by looking at the `ScrollView` component, and how it makes life much simpler by providing native scrolling behavior for wrapped components.

Next, we spent some time implementing buttons with touch feedback. This is another area that's tricky to get right on the mobile web. We learned how to use the `TouchableOpacity`, `TouchableHighlight`, and `Pressable` API components to do this.

Finally, we implemented a generic `Swipeable` component. Swiping is a common mobile pattern, and it allows the user to discover how things work without feeling intimidated.

In the next chapter, we'll learn how to control animation using React Native.